

Dyanamic Programming

Kanishk Goel

24 February, 2026

1 Maximum Weight Independent Set on a Path Graph

Independent Set: A subset of vertices $S \subseteq V$ such that no two vertices in S are adjacent. For path graph, this would mean if $v_i \in S \implies \{v_{i-1}, v_{i+1}\} \cap S = \emptyset$

Problem: MWIS in path graph
Input: An path graph $G = (V, E)$ with n vertices, denoted $v_1, v_2, \dots, v_n$, each having non negative weight w_i Output: Find an independent set where sum of weights is maximised, also find its sum

Let $A[i]$ be the maximum weight for the first i vertices. The Recursion becomes:

$$A[i] = \max(A[i-1], A[i-2] + w_i)$$

Solution
$A[0] \leftarrow 0, A[1] \leftarrow w[1]$ for $i \leftarrow 2$ to n do $A[i] = \max(A[i-1], A[i-2] + w_i)$ return $A[n]$

Go backward from n to reconstruct the set of vertices chosen.

2 Weighted Interval Scheduling

Problem: Weighted Interval Scheduling
Input: set of jobs, with each job having a start time, finish time, and a weight (value/profit). Output: A subset of non-overlapping jobs that maximizes the total weight.

- Let the n intervals be sorted by finish times.
- $OPT[i]$ be the maximum weight for the first i intervals or jobs.
- $p(i)$ be the largest index $j < i$ such that $f_j \leq s_i$.

The Recursion becomes:

$$OPT[i] = \max(OPT[i-1], v_i + OPT[p(i)])$$

Solution
$OPT[0] \leftarrow 0$ // of size $n+1$ for $i \leftarrow 1$ to n do $OPT[i] = \max(OPT[i-1], v_i + OPT[p(i)])$ return $OPT[n]$

Here, the values $p(i)$ have been precomputed using binary search, and the sorting has been done beforehand.

3 0/1 Knapsack Problem

Problem: 0/1 Knapsack
Input: There are n items with weight w_i , value v_i and W is the maximum weight that the knapsack can hold. Output: Find the maximum value that the knapsack can hold.

Let $K[i, w]$ be the maximum value achievable using the first i items with a combined weight of w_i . The Recursion becomes:

$$K[i, w] = \begin{cases} K[i-1, w] & \text{if } w_i > w, \\ \max(K[i-1, w], v_i + K[i-1, w - w_i]) & \text{if } w_i \leq w. \end{cases}$$

Here $w \in [0, W]$ and K is a $2D$ array of size $(n+1) \times (W+1)$

Solution
<pre> for $w \leftarrow 0$ to W do $K[0, w] \leftarrow 0$ for $i \leftarrow 0$ to n do $K[i, 0] \leftarrow 0$ for $i \leftarrow 1$ to n do for $j \leftarrow 1$ to W do if $w[i] > j$ then $K[i, j] \leftarrow K[i - 1, j]$ else $K[i, j] \leftarrow \max(K[i - 1, j], v[i] + K[i - 1, j - w[i]])$ return $K[n, W]$ </pre>

4 Longest Increasing Subsequence

Problem: LIS
Input: Array a, with n numbers Output: Find the longest strictly increasing subsequence in a

4.1 Solution in $O(n^2)$

Define $d[0, 1, \dots, n]$, where $d[i]$ is the length of the longest increasing subsequence ending at index i . Therefore,

$$d[i] = \max \left(1, \max_{\substack{j < i \\ a[j] < a[i]}} (d[j] + 1) \right)$$

maintaining an array $p[i] = j$, where j is the maximal value of $d[j]$ found for index i , also allows us to reconstruct the sequence (Not required but is the easier way).

4.2 Solution in $O(n \log n)$

We define $d[0, \dots, n]$ such that $d[l]$ defines the smallest element at which an increasing subsequence of length l ends.

We go from left to right and find the place to insert $a[i]$ in d using binary search. Hence, the Time Complexity - $O(n \log n)$ and Space Complexity - $O(n)$.

4.2.1 Restoring subsequence

We maintain two auxiliary arrays, one that tells us the index of elements in $d[]$, and an array of ancestors $p[]$ such that $p[i]$ will be the index of the previous element for the optimal subsequence ending at element i .